

A) Simple banking model

.1) Bounds and types

`MAX = 5`

- **What it is:** a constant bound on state growth.
- **Why needed:** your state variables `avail` and `held` otherwise grow without limit due to deposits (+1), giving **infinite state space**. FDR model checking needs a **finite** state space.
- **What you gain:** FDR can fully build the LTS, show structures, check assertions, and won't hang at *Evaluating process*.

`BAL = {0..MAX}`

- **What it is:** a **finite set** of values.
- **Why needed:** channels carrying values must have a finite domain for exploration.
- **Why BAL (or any set name) is better than Nat:**
 - In **FDR 4.2.7 CSP-M**, `Nat` is **not built-in** (so `channel show_balance : Nat` causes *Nat is not in scope*).
 - Even if you define a set yourself, using a clear name like `BAL` avoids confusion with “type systems” from other formalisms.

2) Channel declarations

`channel deposit_small, deposit_large, withdraw, query, clear_large, fraud_withdraw`

- Declares **events**. These are simple event names (no data attached).

`channel show_balance : BAL`

- Declares a **data-carrying channel**: events like `show_balance.0`, `show_balance.1`, ..., `show_balance.5`.
- This is exactly the “channels as families of events” idea.

3) The process header

`ACCOUNT(avail, held) = ...`

- `avail` and `held` are **state parameters** (not channel types).
- FDR treats each distinct pair `(avail, held)` as a **distinct state**.

4) Each branch and what it means

CSP operator used here is external choice `[]` (more later).

Small deposit

```
(avail < MAX & deposit_small -> ACCOUNT(avail+1, held))
```

- **Guard:** only allow if `avail < MAX`.
- Without this guard, `avail` grows forever.
- Transition effect: increments `avail`.

Large deposit

```
(held < MAX & deposit_large -> ACCOUNT(avail, held+1))
```

- Same idea: bounds `held`.
- Large deposits go into “held” not available.

Withdraw

```
(avail > 0 & withdraw -> ACCOUNT(avail-1, held))
```

- Guard prevents negative balances.
- Standard “only withdraw if available funds”.

Query / show balance

```
query -> show_balance!avail -> ACCOUNT(avail, held)
```

- Two-step interaction:
 1. environment triggers `query`
 2. system outputs `show_balance.avail`
- State stays unchanged.

Clear large deposits

```
(held > 0 & avail + held <= MAX & clear_large -> ACCOUNT(avail+held, 0))
```

- Requires held funds exist.
- Also bounded: `avail + held <= MAX` to keep `avail` within domain after transfer.
- Moves all held into available, resets held.

Fraudulent withdrawal

```
(avail > 0 & fraud_withdraw -> ACCOUNT(avail-1, held))
```

- Looks similar to `withdraw`, but semantically represents an **unauthorised path** (policy violation).
- Great for teaching: it enables traces absent from “safe” model.

5) Initialisation

BANK = ACCOUNT (0, 0)

- Defines starting state.

MAIN = BANK

- Tells FDR what to analyse.

B) Explain commands

:

Model inspection / exploration

:processes

- Lists all bound process names (e.g., `MAIN`, `BANK`, `ACCOUNT`).
- Use it to confirm FDR loaded your file.

:graph <PROCESS>

- Shows the **LTS** (Labelled Transition System): states + transitions.
- For your bank, you’ll see nodes labelled like `ACCOUNT (2, 1)` and edges labelled `deposit_small`, `query`, etc.

:probe <PROCESS>

- Interactive “step” exploration: from current state, shows enabled events and next states.
- What you “see”: a tree-like unfolding, very useful for teaching guards.

:structure <PROCESS>

- Shows the **syntactic structure** and compilation summary.
- What you “see”: operator (e.g. External Choice), alphabet, number of states/transitions.

:communication_graph <PROCESS>

- Shows who synchronises with whom — **only meaningful for parallel compositions** (“high-level machines”).
- If your process is a single machine (like `ACCOUNT(0,0)` alone), it will say it cannot display the graph.

Assertions / verification workflow

`:assertions`

- Lists assertions found in the file and (after run) pass/fail status.

`:run <assertion-index-or-name>`

- Runs a particular assertion.
- Typical workflow: `:assertions` → pick index → `:run 0`.

`:counterexample <assertion-index-or-name>`

- If an assertion fails, this prints the **counterexample trace** (a concrete sequence of events causing failure).

`:debug <assertion-index-or-name>`

- Interactive debugging of a failing assertion; helps understand *why* and where it fails.

`:statistics <assertion-index-or-name>`

- Shows stats about the assertion check (states, transitions explored, etc.).
- Important: in FDR4 you must pass an index/name; `:statistics` alone errors.

Environment / utility

`:load <file>`

- Loads a CSP file.

`:reload`

- Reloads the current file (after editing).

`:cd <path>`

- Change working directory.

`:options`

- View/modify tool options.

`:profiling_data`

- Shows profiling data if enabled (performance diagnostics).

`:type <expression>`

- Prints the inferred type of an expression (helpful for channel/set type issues).

`:version`

- Prints FDR version.

`:quit`

- Exit.

C) Assertions

1) Deadlock freedom

Add:

```
assert MAIN :[deadlock free]
```

Meaning:

- In every reachable state, there is at least one enabled event (no deadlock).

What you see:

- `:assertions` shows it
- `:run 0` checks it
- pass/fail output; if fail, `:counterexample 0` gives a deadlocking trace.

2) Refinement

Define a SAFE model (no fraud events), then:

```
assert MAIN [T= SAFE
```

Meaning:

- Every trace of `MAIN` is allowed by `SAFE`.
- If `fraud_withdraw` exists in `MAIN` but not `SAFE`, it should **fail**.
- `:counterexample` will show a trace proving the extra behaviour.

D) Concurrency — what it is in CSP/FDR, what “choice” makes it visible, and what FDR shows

1) What concurrency means in CSP

Concurrency is expressed by **parallel operators**, e.g.:

- Interleaving (no synchronisation):
 - $P \parallel Q$ (events interleave freely)
- Synchronising parallel:
 - $P [A] Q$ (must synchronise on events in set A)

This is what turns your model into a “high-level machine” and enables:

- `:communication_graph`

2) Which “choice” shows something?

Two key operators show different things:

External choice $P [] Q$

- **Environment** picks the first event.
- You “see” it as branching in LTS/probe.

Parallel composition $P [A] Q / P \parallel Q$

- You “see” interleavings and synchronisations.
- You now can use **communication graph**.

So:

- $[]$ gives “branching due to environment choice”
- $\parallel / [A]$ gives “concurrency structure and synchronisation”

E) Advanced scenario script — ATMs + concurrency (clean CSPM you can use)

This is the “clear script” you asked for: an **ATM/Bank/FraudCheck concurrent model** that will:

- show concurrency in the LTS
- allow :communication_graph SYSTEM
- remain finite with MAX
- demonstrate external choice, channels, and synchronisation

```

MAX = 5
BAL = {0..MAX}

-- shared channels between components
channel deposit_small, deposit_large, withdraw, query
channel clear_large, fraud_withdraw
channel show_balance : BAL

-- ATM: the user interface (environment-facing)
ATM =
  deposit_small -> ATM
[]
  deposit_large -> ATM
[]
  withdraw      -> ATM
[]
  query         -> ATM
[]
  fraud_withdraw -> ATM

-- Bank core: stateful account logic
BANK(avail, held) =
  (avail < MAX & deposit_small -> BANK(avail+1, held))
[]
  (held < MAX & deposit_large  -> BANK(avail, held+1))
[]
  (avail > 0 & withdraw        -> BANK(avail-1, held))
[]
  query -> show_balance!avail -> BANK(avail, held)
[]
  (held > 0 & avail+held <= MAX & clear_large -> BANK(avail+held,
0))
[]
  (avail > 0 & fraud_withdraw -> BANK(avail-1, held))

-- Fraud service: runs "in parallel" and can clear or allow fraud
withdrawal
FRAUD_SERVICE =
  clear_large  -> FRAUD_SERVICE
[]
  fraud_withdraw -> FRAUD_SERVICE

-- System composition:
-- ATM synchronises with BANK on user actions,
-- BANK synchronises with FRAUD_SERVICE on fraud-related events.
SYSTEM =
  (ATM [| {|deposit_small, deposit_large, withdraw, query,
fraud_withdraw|} |] BANK(0,0))
  [| {|clear_large, fraud_withdraw|} |]
  FRAUD_SERVICE

MAIN = SYSTEM

-- Assertions (examples)
assert MAIN :[deadlock free]

```

What to run in FDR

- `:graph MAIN` → shows interleavings/synchronisations
- `:communication_graph MAIN` → now works (because MAIN is parallel composition)
- `:probe MAIN` → step through which component is participating in which event

F) Theory explanations

1) Channels and data:

$$c.T = \{c.x \mid x:T\} \subset \Sigma$$

- A channel c with type/set T denotes the set of all events $c.x$.
- This is how CSP represents message passing while remaining event-based.

2) Input and output

Input:

$$c?x:T \rightarrow P(x)$$

- Means the process will accept **any** $x \in T$.
- Semantically: an **external choice** over all $c.x$.

Output:

$$c!v \rightarrow P$$

- Means the process commits to outputting **that specific** value v .

Copy example:

$$\text{Copy} = \text{left}?x:T \rightarrow \text{right}!x \rightarrow \text{Copy}$$

- Receives any value on `left`
- Outputs the same value on `right`
- Repeats forever
- Great for demonstrating “input introduces branching; output preserves value”.

3) External choice $P \sqcup Q$

Key idea:

- The **environment** selects the first event by choosing one that is offered.
- If the chosen first event belongs only to P , the system behaves like P .
- If only to Q , behaves like Q .

Example:

$(a \rightarrow A) [] (b \rightarrow B)$

- Environment chooses $a \rightarrow$ then system becomes A
- Environment chooses $b \rightarrow$ then system becomes B

4) Law: $P = P [] \text{STOP}$

- STOP offers **no events**.
- External choice offers the environment the union of initials of both processes.
- Since STOP offers nothing, adding it gives no additional initial event options.
- Therefore the choice behaves exactly like P .

Arithmetic analogy:

Adding zero doesn't change a number: $x + 0 = x$

Here, STOP is like "adding no possibilities".

Below is a **complete concurrent CSP-M model** for a *more complex banking scenario with ATMs* where:

- You have **multiple ATMs** operating **in parallel** (so deposits/withdrawals can occur "at the same time" in the model, i.e. via interleavings).
- All ATMs access a **shared bank core** (single account state).
- Operations are **parameterised by ATM id** (so you can see which ATM did what).
- It supports **balance query** returning a value (via a channel).
- It stays **finite** for FDR by using MAX and bounded domains.
- It is a true **parallel composition**, so `:communication_graph` will work.

Then I'll explain **where concurrency is added** and how to reason about "parallel deposit and withdrawal".

1) Full code: Concurrent bank + multiple ATMs

```
-- =====
-- Concurrent ATM banking model (FDR4 / CSP-M)
-- Shared bank core, multiple ATMs in parallel
-- =====
```

```

-- ----- Finite bounds (required for model checking) -----
MAX = 5
BAL = {0..MAX}

-- Number of ATMs (small is better for state space)
ATMID = {1..2}

-- ----- Channels (operations tagged by ATM id) -----
channel deposit_small, deposit_large, withdraw, query,
fraud_withdraw, clear_large : ATMID
channel show_balance : ATMID.BAL

-- Helper: total money stays within MAX (simple bound discipline)
withinMax(avail, held) = (avail >= 0 and held >= 0 and avail <= MAX
and held <= MAX and avail + held <= MAX)

-- ----- BANK CORE (shared state machine) -----
-- avail = available funds
-- held = held funds (large deposits pending clear)
BANK(avail, held) =

  -- Small deposit: immediate availability
  ([ a:ATMID @
    (withinMax(avail, held) and avail < MAX) &
    deposit_small.a ->
      BANK(avail + 1, held)
  ])

  []

  -- Large deposit: goes to held
  ([ a:ATMID @
    (withinMax(avail, held) and held < MAX and avail + held < MAX) &
    deposit_large.a ->
      BANK(avail, held + 1)
  ])

  []

  -- Withdraw: only if available > 0
  ([ a:ATMID @
    (withinMax(avail, held) and avail > 0) &
    withdraw.a ->
      BANK(avail - 1, held)
  ])

  []

  -- Query: respond with current avail to the requesting ATM
  ([ a:ATMID @
    query.a ->
      show_balance.a!avail ->
        BANK(avail, held)
  ])

  []

  -- Clear held funds (e.g., after fraud check)
  ([ a:ATMID @
    (withinMax(avail, held) and held > 0 and avail + held <= MAX) &
    clear_large.a ->

```

```

        BANK(avail + held, 0)
    )

[]

-- Fraud withdrawal: reduces avail without policy checks beyond
avail>0
([[] a:ATMID @
  (withinMax(avail, held) and avail > 0) &
  fraud_withdraw.a ->
    BANK(avail - 1, held)
])

-- ----- ATM process (environment-facing interface) -----
-- Each ATM can initiate actions repeatedly
ATM(a) =
  deposit_small.a -> ATM(a)
[]
  deposit_large.a -> ATM(a)
[]
  withdraw.a      -> ATM(a)
[]
  query.a         -> ATM_WAIT_BALANCE(a)
[]
  fraud_withdraw.a -> ATM(a)
[]
  clear_large.a   -> ATM(a)

ATM_WAIT_BALANCE(a) =
  show_balance.a?b:BAL -> ATM(a)

-- ----- SYSTEM composition -----
-- All ATMs interleave with each other (true concurrency via
interleaving),
-- and they synchronise with the shared BANK on matching events.
ATMS =
  ||| a:ATMID @ ATM(a)

-- Synchronisation set: all ATM-tagged operations plus balance
responses
SYNC =
  { deposit_small.a, deposit_large.a, withdraw.a, query.a,
    fraud_withdraw.a, clear_large.a, show_balance.a.b
    | a <- ATMID, b <- BAL }

SYSTEM =
  ATMS [| SYNC |] BANK(0, 0)

MAIN = SYSTEM

-- ----- Example assertion -----
assert MAIN :[deadlock free]

```

2) Where concurrency is added (the exact lines)

Concurrency is introduced **only** by parallel composition operators. In this model you have **two levels**:

(A) Multiple ATMs running concurrently

```
ATMS = ||| a:ATMID @ ATM(a)
```

- `|||` is **interleaving parallel**: ATMs run “at the same time” in the model.
- It means: actions from ATM1 and ATM2 can occur in any order.
- This is how CSP models concurrency: by **all possible interleavings**.

If you increase `ATMID` to `{1..3}`, you get three ATMs in parallel (state explosion grows fast).

(B) ATMs synchronising with the shared bank core

```
SYSTEM = ATMS [| SYNC |] BANK(0, 0)
```

- `[| SYNC |]` is **synchronising parallel**.
- It forces the ATMs and BANK to perform events together, when the event is in `SYNC`.
- This models the fact that:
an ATM cannot “withdraw” unless the bank core simultaneously performs `withdraw.a` and updates state.

This is the *correct* way to model a shared server.

3) “Parallel deposit and withdrawal” — what it means in CSP

In CSP/FDR you never get “true simultaneous” events in one step. Instead you get:

- all possible **interleavings** of concurrent components

So “ATM1 deposits while ATM2 withdraws” becomes two possible traces:

1. deposit first:

```
<deposit_small.1, withdraw.2>
```

2. withdraw first:

```
<withdraw.2, deposit_small.1>
```

Both are explored by FDR.

That's exactly why concurrency is valuable: you automatically cover all schedules.

4) Why we tag events with ATM id

Without tagging:

```
deposit_small  
withdraw
```

two ATMs would be indistinguishable in traces. With tagging:

```
deposit_small.1  
withdraw.2
```

you can:

- show which ATM did what
- talk about race-like behaviour and scheduling
- generate counterexamples that name the source ATM

5) Why $\text{MAX} + \text{BAL}$ are still required in the concurrent model

Concurrency makes state explosion worse. So bounds are even more important:

- MAX bounds **state variables** (`avail, held`) $\rightarrow$ finite LTS
- BAL bounds **data-carrying events** (`show_balance.a.b`) $\rightarrow$ finite alphabet

Without MAX , FDR may hang again at “Evaluating process”.

6) Traces in FDR

Communication graph will now work:

```
:communication_graph MAIN
```

Because `MAIN` is a parallel composition: `ATMS [| SYNC |] BANK`.

Probe now shows interleavings across ATMs:

```
:probe MAIN
```

You should see enabled actions like:

- `deposit_small.1`
- `deposit_small.2`
- `withdraw.1` (when `avail>0`)
- `query.2 then show_balance.2.0`

Structure shows high-level machine:

```
:structure MAIN
```

You'll see it as a high-level machine composed by parallel operators.

7) If you want “locking / atomicity”

Right now, operations are atomic because they synchronise with BANK. If you want to model a more realistic two-step protocol (request/response), you can split each operation into:

- `req_withdraw.a then resp_withdraw_ok.a / resp_withdraw_fail.a`

That increases realism and is great for teaching security/authentication flows.

Analysis of the following part

```
||| a:ATMID @ ATM(a)
```

This is called **replicated interleaving parallel composition**.

I'll explain it in layers:

1. The operator `|||`
2. The replication `a:ATMID @`
3. What the whole expression *means semantically*
4. What it gives you in FDR (what you can see)
5. A concrete intuition with ATMs

1The operator `|||` (interleaving parallel)

In CSP:

$P \parallel Q$

means:

“Run process P and process Q **in parallel**,
with **no required synchronisation** between them.”

Key properties:

- Events from P and Q can occur in **any order**
- There is **no shared state**
- This models **independent concurrent components**

This is how CSP represents *concurrency*: by **all possible interleavings** of events.

2Replication: $a:ATMID @ \dots$

The syntax:

$a:ATMID @ ATM(a)$

means:

“For every value a in the set $ATMID$, create a copy of $ATM(a)$.”

So if:

$ATMID = \{1, 2, 3\}$

then:

$ATM(1), ATM(2), ATM(3)$

are three separate processes.

3Putting it together: replicated interleaving

Now combine both ideas:

$\parallel a:ATMID @ ATM(a)$

This is **syntactic** for:

$ATM(1) \parallel ATM(2) \parallel ATM(3)$

Meaning:

“Run ATM 1, ATM 2, and ATM 3 **concurrently**, allowing any interleaving of their actions.”

4 What this means semantically

- Each $\text{ATM}(a)$ has its **own control flow**
- They do **not** automatically coordinate with each other
- FDR explores **all possible schedules**, e.g.:

```
deposit_small.1  
withdraw.2  
deposit_small.3  
query.1
```

or

```
query.3  
deposit_small.3  
withdraw.1
```

5 Why this is the right way to model ATMs

In the real world:

- ATM 1, ATM 2, ATM 3 operate independently
- Any of them can initiate actions at any time

This is **exactly** what `||| a:ATMID @ ATM(a) captures`.

6 What you will see in FDR

In `:probe MAIN`

From the initial state, you’ll see enabled events like:

```
insert_card.1  
insert_card.2  
insert_card.3
```

Meaning:

- any ATM may act next

After ATM 1 logs in, you might see:

```
deposit_small.1  
withdraw.1  
insert_card.2  
insert_card.3
```


All interleavings are explored.

In :structure MAIN

You will see:

- a **parallel operator**
- replicated structure
- confirmation that this is a **high-level machine**

In :communication_graph MAIN

You will see:

- multiple ATM components
- edges from each ATM to the shared bank server
- labels showing which events synchronise

7 Why not use [] (external choice) here?

This is a **very important distinction**.

This would be wrong:

```
[] a:ATMID @ ATM(a)
```

That would mean:

- the **environment chooses one ATM**
- only that ATM exists
- no concurrency at all

Correct:

```
||| a:ATMID @ ATM(a)
```

Because:

- all ATMs exist
- all can act
- actions interleave

8 Arithmetic analogy

`[]` is like **choosing one option**

- `|||` is like **adding independent processes**

If external choice is like:

```
choose one road
```

Then interleaving parallel is like:

```
many cars driving independently on different roads
```

Adding parallelism in the simple banking scenario

1 Why your current model has NO parallelism

Your current system is:

```
MAIN = ACCOUNT(0, 0)
```

and `ACCOUNT(avail, held)` is a **single process** using only:

```
[] -- external choice
```

This means:

- At each step, **exactly one event happens**
- The *environment* chooses which event
- There are **no concurrent components**

Even though deposits and withdrawals *look independent*, they are just **alternative transitions**, not parallel ones.

This is a **sequential state machine**, not a concurrent system.

2 How to add parallelism correctly (design idea)

To model *parallel deposit and withdrawal*, we need:

- **Two (or more) independent processes** that can act concurrently
- A **shared account** that synchronises with them

Design pattern (standard CSP):

```
(DEPOSITOR ||| WITHDRAWER) [| {deposit, withdraw} |] ACCOUNT
```

- DEPOSITOR and WITHDRAWER run **in parallel**
- ACCOUNT is the **shared resource**
- Synchronisation ensures atomic updates

This is exactly how you model:

- concurrent clients
- shared databases
- bank backends

3Full code: adding parallelism to the model

Complete concurrent version

```
-- =====
-- Finite bounds for model checking
-- =====
MAX = 5
BAL = {0..MAX}

-- =====
-- Events / channels
-- =====
channel deposit_small, deposit_large, withdraw, query, clear_large,
fraud_withdraw
channel show_balance : BAL

-- =====
-- Bank account core (stateful)
-- =====
ACCOUNT(avail, held) =

  (avail < MAX & deposit_small ->
    ACCOUNT(avail + 1, held))

  []

  (held < MAX & deposit_large ->
    ACCOUNT(avail, held + 1))

  []

  (avail > 0 & withdraw ->
    ACCOUNT(avail - 1, held))

  []

  query ->
    show_balance!avail ->
    ACCOUNT(avail, held)

  []
```

```

(held > 0 & avail + held <= MAX & clear_large ->
  ACCOUNT(avail + held, 0))

[]

(avail > 0 & fraud withdraw ->
  ACCOUNT(avail - 1, held))

-- =====
-- Concurrent clients
-- =====

-- A process that repeatedly makes deposits
DEPOSITOR =
  deposit_small -> DEPOSITOR
[]
  deposit_large -> DEPOSITOR

-- A process that repeatedly withdraws
WITHDRAWER =
  withdraw -> WITHDRAWER

-- =====
-- Parallel system
-- =====

CLIENTS =
  DEPOSITOR ||| WITHDRAWER

SYSTEM =
  CLIENTS [| {|deposit_small, deposit_large, withdraw|} |]
ACCOUNT(0,0)

MAIN = SYSTEM

-- =====
-- Example assertion
-- =====
assert MAIN :[deadlock free]

```

4Where parallelism is added

Parallel deposit vs withdrawal

```

CLIENTS =
  DEPOSITOR ||| WITHDRAWER

```

This is **interleaving parallel**:

- Depositor and withdrawer run concurrently
- Their actions can interleave in any order

This is where **parallelism first appears**.

Shared state synchronisation

```
SYSTEM =  
  CLIENTS [| {|deposit_small, deposit_large, withdraw|} |]  
  ACCOUNT(0,0)
```

This ensures:

- A deposit or withdrawal **only happens when the account also participates**
- State updates are **atomic**
- No race conditions on `avail`

This is *exactly* how shared memory is modelled in CSP.

5 What “parallel deposit and withdrawal” is

In CSP, this means **all interleavings are explored**.

FDR will consider traces like:

```
<deposit_small, withdraw, deposit_large>
```

and also:

```
<withdraw, deposit_small, deposit_small>
```

This models:

- deposit and withdrawal happening “at the same time”
- without assuming a fixed scheduling order

6 FDR traces

```
:structure MAIN
```

- Shows a **parallel composition**
- High-level machine

```
:communication_graph MAIN
```

- Shows CLIENTS ↔ ACCOUNT synchronisation
- Depositor and withdrawer both connected to ACCOUNT

```
:probe MAIN
```

- From the initial state, both `deposit_*` and `withdraw` may be enabled
- But `withdraw` only appears once `avail > 0`

